

Model Extraction (Model Stealing) Attack

Stealing the Functionality of a Black-Box Image Classifier

CSE 406 • Cyber Security Sessional — **Final Project Report**

Abrar Zahin Raihan (ID: 2105009)

Md. Mahmud Hasan (ID: 2105027)

Group 14

September 19, 2026

Abstract

We implemented and measured a functionality-stealing attack against a black-box image classifier, following the design we submitted on 11 September 2026. The victim is a CIFAR-adapted ResNet-18 trained on CIFAR-10 to 94.33% test accuracy and served behind a FastAPI prediction endpoint. The attacker queries that endpoint, logs the returned predictions, and trains a substitute offline. The substitute is a four-layer CNN with 2.36M parameters, 4.7 times smaller than the victim, trained from random initialisation. Sweeping the query budget from 5,000 to 50,000, across two transfer pools (in-distribution CIFAR-10 and out-of-distribution CIFAR-100) and two supervision signals (probability vectors and top-1 labels), we recover a substitute reaching 86.62% accuracy and 87.44% agreement with the victim in-domain, and 74.29% accuracy using query images from a different label space. Those are 91.8% and 78.8% of the victim’s own accuracy, obtained without its weights, architecture, or any ground-truth label. We then evaluated four defenses. The result contradicted our design-report hypothesis. Reducing output granularity by rounding probabilities, returning only the top-1 probability, or returning only the top-1 label changed the attacker’s accuracy by between +0.09 and −3.82 points, mostly inside a measured run-to-run noise floor of 3.24 points. Only output perturbation had a substantial effect, cutting the substitute from 74.99% to 51.66% at 50,000 queries and raising the attacker’s effective query cost by 2.39 times. Inspecting our own server code explains why: under the top-1 and rounding policies, the soft-label loss reduces algebraically to hard-label cross-entropy, so those defenses hand the attacker a signal it already had.

1 Introduction and Threat Model

Model extraction, or model stealing, is an attack in which an adversary holding only black-box query access to a trained model reconstructs a functionally equivalent copy [1, 2, 3]. The attacker never observes the victim’s weights, architecture, or training data. It sees only what the model returns for inputs of its own choosing [1, 2]. We write f_V for the victim model and f_S for the substitute we train.

Following Jagielski et al. [3] we separate two success criteria and report both:

- **Accuracy.** How well f_S performs the underlying task on real data.
- **Fidelity.** How often f_S agrees with f_V , including on inputs where f_V is itself wrong. A high-fidelity copy reproduces the victim’s mistakes.

These objectives conflict for an imperfect victim, since a maximally accurate substitute should correct the victim’s errors while a maximally faithful one should reproduce them [3]. Our victim is 94.33% accurate, so the two metrics stay close, but the gap between them carries information and we analyse it in Section 5.3.

Our attack reproduces functionality stealing in the style of Knockoff Nets [2]. We query the victim with images drawn from any convenient pool, treat the returned predictions as labels, and train our own network on those pairs. We need neither the victim’s training data nor its architecture [2]. Tramèr et al. [1] showed the same idea works against machine-learning-as-a-service models, in simple cases by direct equation solving, and Knockoff Nets extends it to deep networks through a query-and-imitate

loop [2]. The same substitute-training primitive was used earlier by Papernot et al. [4] as a stepping stone to black-box evasion.

Adversary model. The adversary has query access to f_V and nothing else. It does not know the victim’s architecture, hyperparameters, weights, training set, or the semantics of the output classes. It has a fixed budget B of queries and may choose the query inputs freely, including inputs from a completely different distribution than the victim’s training data. This is the standard model-extraction threat model of Tramèr et al. [1], situated within the taxonomy of adversarial goals, knowledge, and capabilities of Jagielski et al. [3]. In that taxonomy ours is a theft adversary: its motive is to obtain the model’s functionality without paying its training cost.

Claim under test. A black-box classifier can be duplicated cheaply through its prediction API [1, 2], and restricting what that API returns does not by itself prevent it.

2 System Design and Components

Because this is an application-layer attack, the topology is minimal: a query channel between two hosts. Figure 1 shows the arrangement. Both roles can live on one machine, with the victim as a local server on `localhost` and the attacker as a script, or on two hosts on the same LAN. In our experiments the sweep driver launches the victim as a `uvicorn` subprocess on `127.0.0.1:8000` and polls `/health` until it responds, then queries it over HTTP exactly as a remote client would.

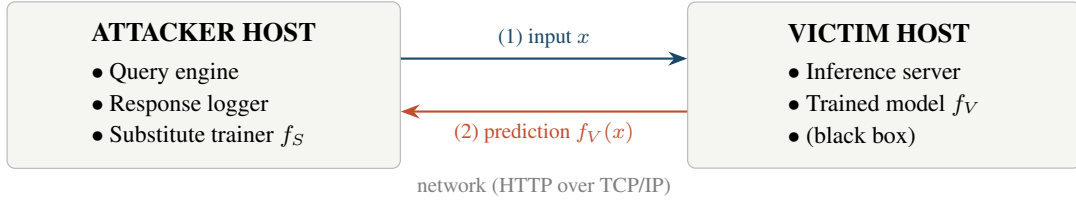


Figure 1: Attack topology. (1) The attacker sends a chosen input x to the victim’s prediction API. (2) The victim returns its prediction $f_V(x)$, a label or a probability vector. The attacker treats each $(x, f_V(x))$ pair as a training example for f_S .

2.1 Components

Component	Module	Role
Victim model f_V	<code>victim_train.py</code>	ResNet-18 trained on CIFAR-10, then never inspected.
Inference server	<code>server.py</code>	Wraps f_V , applies the output policy, returns predictions.
Attacker query engine	<code>attack.py</code>	Selects pool images and sends them within budget B .
Response logger	<code>attack.py</code>	Caches every $(x_i, f_V(x_i))$ triple across budgets.
Substitute trainer	<code>substitute.py</code>	Trains f_S offline on the logged pairs.
Evaluator	<code>evaluate.py</code>	Scores accuracy and fidelity on the CIFAR-10 test split.

Table 1: Components of the attack and the module implementing each. Relative to the design report, the inference server gained one responsibility: applying the configured output policy before responding (Section 6).

One detail of the query engine matters for reading the budget curves. `TransferQuerier` fixes a single shuffled ordering of the transfer pool under seed 0 and caches answers, so the budgets are nested: the 5,000 queries used at $B = 5,000$ are the first 5,000 of the 10,000 used at $B = 10,000$, and so on. Each budget pays only for its new queries. Since both transfer pools hold exactly 50,000 images, the largest budget consumes the entire pool.

2.2 Timing of the query loop

Under normal operation the prediction API serves a single request: an honest client sends one input and consumes one answer. The attack reuses the same request and response messages and changes only the intent behind them. The attacker issues inputs in batches of 256, logs every returned prediction, and repeats until the budget is spent. Only then does it move offline to train the substitute, with no further queries. Figure 2 shows the loop.



Figure 2: Attack timing diagram. The attacker sends batches and logs the returned predictions until the budget is exhausted, then trains f_S entirely offline. The offline phase is free: once the transfer set is collected, the attacker can retrain f_S any number of times, with different architectures or losses, at no further query cost.

3 Message Structure

The attack operates at the application layer, so there is no custom L2/L3 frame to craft. The Ethernet, IP, and TCP headers are ordinary and untouched. What matters is the request and response carrying the model input and output. Figure 3 places that payload inside the surrounding protocol stack.

Images travel as raw uint8 pixels in C-order, base64-encoded, with an explicit shape field. The server performs CIFAR normalisation itself, so the attacker only ever transmits pixels and never a pre-normalised tensor. This mirrors a realistic prediction API, where clients upload images and the service preprocesses them, and it removes the most common source of silent mismatch between attacker and victim.

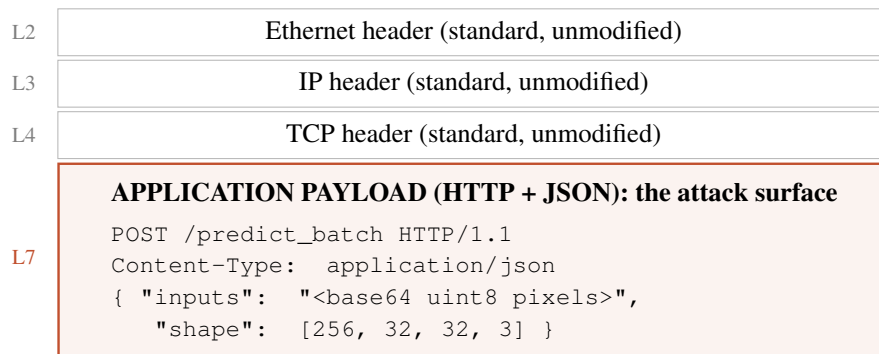


Figure 3: Protocol stack for one batched query. The lower layers are ordinary and left untouched; the entire attack lives in the application-layer payload.

Response (victim → attacker):

```

HTTP/1.1 200 OK
Content-Type: application/json
{ "labels": [3, 8, 0, ...],
  
```

```
"probabilities": [[0.02, 0.05, 0.11, 0.71, ...], ...] }
                // present, rounded, truncated, perturbed, or omitted
                // entirely, depending on the active output policy
```

The `labels` field is always present. The `probabilities` field is what the defenses in Section 6 manipulate, and it is the field separating the two access levels the reference papers distinguish [1, 3]. A full probability vector carries the class-similarity structure that knowledge distillation exploits [5]; a bare label does not. Section 6.3 reports how much that distinction is worth in practice, and the answer is less than we predicted.

4 Experimental Setup

4.1 Victim

The victim is a CIFAR-adapted ResNet-18 [10]: a 3×3 stem at stride 1 with no initial max-pool, four stages of two `BasicBlocks` at widths 64, 128, 256 and 512, and a linear head, totalling 11,173,962 parameters. It was trained on the full CIFAR-10 [9] training set of 50,000 images for 60 epochs with SGD (learning rate 0.1, Nesterov momentum 0.9, weight decay 5×10^{-4}), a cosine-annealed schedule, batch size 128, and standard augmentation consisting of a 4-pixel reflect-padded random crop and a horizontal flip. Its accuracy on the 10,000-image CIFAR-10 test split is **94.33%**.

That figure is recomputed by the evaluator from the loaded checkpoint on every run, and it is what appears in the `victim_accuracy` column of both results files. It should be distinguished from the `test_acc` value stored inside the checkpoint, which `victim_train.py` records as the best accuracy seen across epochs while saving the weights from the final epoch. Those two numbers need not coincide, and only the checkpoint’s stored value feeds the reference line in the driver’s own plots. Every victim accuracy in this report is the recomputed one.

After training, the model is served by `server.py` behind FastAPI and treated as a black box. No later stage loads its weights except the evaluator, which needs f_V ’s predictions to compute fidelity and is not part of the attack.

4.2 Transfer pools

We compared two pools of query images, which is the central axis of the experiment:

- **In-distribution (CIFAR-10 train split).** The attacker queries with the same 50,000 images the victim was trained on, but receives only the victim’s predictions and never the ground-truth labels. Evaluation happens on the disjoint test split, so no answer key leaks into the transfer set. This is the best case for the attacker and serves as an upper bound.
- **Out-of-distribution (CIFAR-100 train split).** 50,000 images from CIFAR-100 [9], which shares the 32×32 format but whose class semantics barely overlap with CIFAR-10. This demonstrates the central claim of Knockoff Nets [2], that the attacker does not need the victim’s data, and matches the non-problem-domain result of Correia-Silva et al. [8].

4.3 Budgets, losses, and the substitute

We swept four query budgets, $B \in \{5,000, 10,000, 20,000, 50,000\}$. The largest budget equals the size of the victim’s training set, which makes the comparison directly interpretable: at $B = 50,000$ the attacker has issued one query per victim training image while holding no ground-truth label.

Two supervision signals were used, matching the two access levels in Section 3:

- **Soft.** Cross-entropy against the victim’s returned probability vector, $\mathcal{L} = -\sum_c p_V(c) \log \text{softmax}(z)_c$, which equals $\text{KL}(p_V \parallel \text{softmax}(z))$ up to a constant. No distillation temperature is applied. A temperature would require the victim’s logits, and the API returns probabilities only, so this is distillation at $T = 1$ on post-softmax outputs.

- **Hard.** Standard cross-entropy on the `argmax` label, the label-only setting of [1, 3].

The substitute is `SmallCNN`, a four-layer convolutional network with 2,360,906 parameters: two 3×3 convolutions at width 64, a max-pool, two at width 128, a second max-pool, then a 256-unit fully connected layer with dropout 0.5 and a linear head. It comes from a different and weaker family than the victim, 4.7 times smaller by parameter count, and is trained from random initialisation for 40 epochs with SGD (learning rate 0.1, Nesterov momentum 0.9, weight decay 5×10^{-4}), cosine annealing, and batch size 256, with the same crop-and-flip augmentation applied to the queried images. Following Knockoff Nets [2], the substitute is not required to match the victim’s architecture. Section 7 argues that this capacity gap is the main reason our accuracy ceiling sits where it does.

4.4 Metrics

Both metrics are evaluated on the clean CIFAR-10 test split, D_{test} , which the attacker never queries and never sees labels for:

$$\text{Accuracy} = \frac{1}{|D_{\text{test}}|} \sum_{x \in D_{\text{test}}} \mathbf{1}[f_S(x) = y_{\text{true}}(x)], \quad (1)$$

$$\text{Fidelity} = \frac{1}{|D_{\text{test}}|} \sum_{x \in D_{\text{test}}} \mathbf{1}[f_S(x) = f_V(x)]. \quad (2)$$

We also report the extraction ratio, $\text{Accuracy}(f_S)/\text{Accuracy}(f_V)$, which expresses the result as a fraction of the victim’s own capability.

4.5 Reproducing the runs

```
make victim ARCH=resnet18 EPOCHS=60 # trains and saves checkpoints/victim.pt
make main SUB_ARCH=smallcnn # -> results/main_results.csv
make defense SUB_ARCH=smallcnn # -> results/defense_results.csv
```

Stack: Python, PyTorch, torchvision, FastAPI, as planned. All results in this report are computed from `main_results.csv` (16 runs) and `defense_results.csv` (20 runs).

5 Results: The Extraction Attack

5.1 The budget curve

Figure 4 is the central result of the project: accuracy and fidelity against query budget, for both transfer pools and both losses. Table 2 gives the same data numerically.

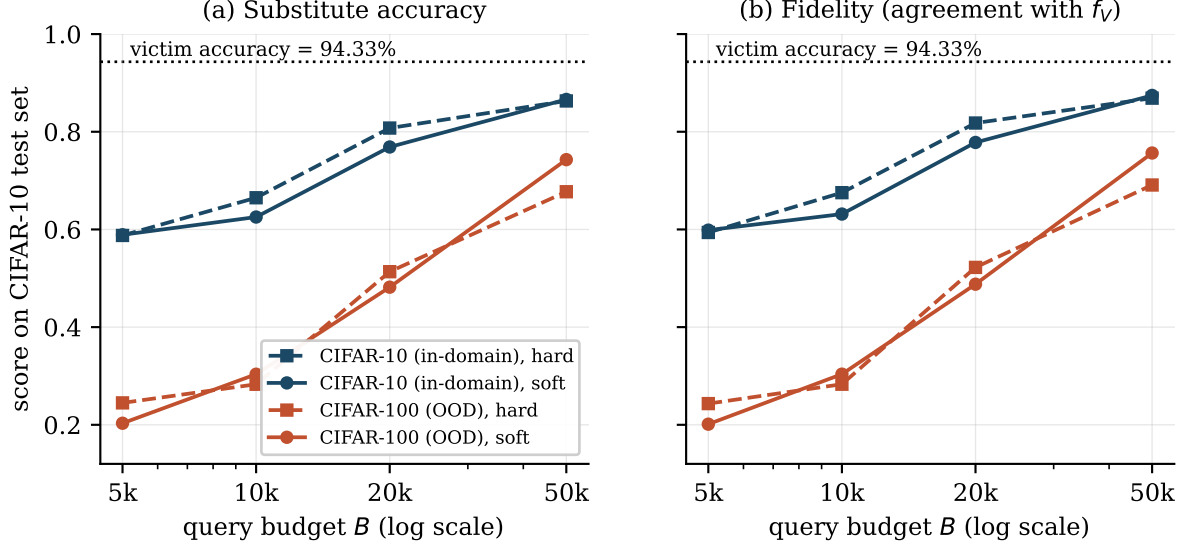


Figure 4: Budget sweep. (a) Substitute accuracy on the CIFAR-10 test split. (b) Fidelity, the fraction of test inputs on which f_S and f_V return the same label. No curve has flattened by 50,000 queries, so every number here is a lower bound on what a richer adversary would achieve.

Transfer pool	Loss	Budget B	Accuracy	Fidelity	Extraction ratio
CIFAR-10 (in-domain)	soft	5,000	58.91%	59.86%	0.625
CIFAR-10 (in-domain)	soft	10,000	62.54%	63.15%	0.663
CIFAR-10 (in-domain)	soft	20,000	76.88%	77.81%	0.815
CIFAR-10 (in-domain)	soft	50,000	86.62%	87.44%	0.918
CIFAR-10 (in-domain)	hard	5,000	58.77%	59.38%	0.623
CIFAR-10 (in-domain)	hard	10,000	66.49%	67.53%	0.705
CIFAR-10 (in-domain)	hard	20,000	80.76%	81.79%	0.856
CIFAR-10 (in-domain)	hard	50,000	86.31%	86.88%	0.915
CIFAR-100 (OOD)	soft	5,000	20.32%	20.13%	0.215
CIFAR-100 (OOD)	soft	10,000	30.36%	30.37%	0.322
CIFAR-100 (OOD)	soft	20,000	48.18%	48.79%	0.511
CIFAR-100 (OOD)	soft	50,000	74.29%	75.65%	0.788
CIFAR-100 (OOD)	hard	5,000	24.49%	24.34%	0.260
CIFAR-100 (OOD)	hard	10,000	28.30%	28.31%	0.300
CIFAR-100 (OOD)	hard	20,000	51.35%	52.23%	0.544
CIFAR-100 (OOD)	hard	50,000	67.72%	69.10%	0.718

Table 2: Full extraction results. Victim accuracy is 94.33% in every row. Extraction ratio is substitute accuracy divided by victim accuracy.

The attack works. At 50,000 in-domain queries the substitute reaches 86.62% accuracy and agrees with the victim on 87.44% of test inputs, recovering 91.8% of the victim’s capability. It does so with no ground-truth labels: every training signal arrived through the prediction API. This reproduces the core claim of [1, 2] end to end. An organisation that paid to collect and annotate CIFAR-10 and to tune a 94%-accurate ResNet-18 has, by exposing a `/predict` endpoint, given away most of that investment to anyone willing to issue 50,000 requests against a network less than a quarter of its size.

The attacker does not need the victim’s data. The CIFAR-100 curves are the more significant result. Querying entirely with images from a different label space still yields a 74.29% substitute, 78.8% of the victim’s accuracy. The victim’s decision boundaries are legible from probes unrelated to its task,

which confirms the central observation of Knockoff Nets [2] and the non-problem-domain result of [8]. Practically, this rules out the most intuitive defense a service operator might reach for. Restricting which images may be submitted does not protect the model, because the attacker does not need the operator’s kind of image.

Out-of-distribution data is not free. The CIFAR-100 curves sit below the CIFAR-10 curves at every budget, and the gap is largest at small budgets: 38.6 points at $B = 5,000$ for the soft loss, closing as the budget grows. Interpolating along the in-domain curve, the 74.29% that CIFAR-100 reaches at 50,000 queries is reached in-domain at roughly 17,600 queries. Out-of-distribution querying therefore costs the attacker about 2.8 times more queries for the same result. The attacker’s ignorance of the victim’s data distribution is worth something to the defender, but it is worth a constant factor rather than a barrier.

5.2 Soft labels versus hard labels

This comparison produced the least tidy result in the project. The intuition from distillation [5] is that full probability vectors should beat bare labels, because the mass on non-argmax classes tells the student which classes the teacher considers similar. Our data supports that in one regime only, as Table 3 shows.

Transfer pool	B	Soft	Hard	Soft – Hard
CIFAR-10	5,000	58.91%	58.77%	+0.14
CIFAR-10	10,000	62.54%	66.49%	−3.95
CIFAR-10	20,000	76.88%	80.76%	−3.88
CIFAR-10	50,000	86.62%	86.31%	+0.31
CIFAR-100	5,000	20.32%	24.49%	−4.17
CIFAR-100	10,000	30.36%	28.30%	+2.06
CIFAR-100	20,000	48.18%	51.35%	−3.17
CIFAR-100	50,000	74.29%	67.72%	+6.57

Table 3: Substitute accuracy by supervision signal, in percentage points. The measured run-to-run noise floor (Section 6.2) is up to 3.24 points at small budgets and 0.70 points at $B = 50,000$, so only the final row is clearly outside noise.

Six of the eight differences fall within, or barely outside, the run-to-run variation measured independently in Section 6.2, so we claim no winner in those regimes. The defensible difference is the last row: at $B = 50,000$ with out-of-distribution queries, soft labels beat hard labels by 6.57 points, more than nine times the 0.70-point noise observed at that budget.

That pattern is interpretable. Soft labels help where the label alone is least informative. For an in-distribution image the victim is confident and its output is nearly one-hot, so the probability vector carries little beyond its `argmax` and discarding it loses almost nothing. For a CIFAR-100 image of an object the victim has no class for, the victim is forced into a diffuse response, and that diffuseness is the informative part: it reports how the image sits relative to several decision boundaries at once. Collapsing it to `argmax` discards the most useful part of the answer. For an attacker, the value of confidence values rises as the query pool drifts further from the victim’s training distribution, which is the setting a realistic attacker operates in.

5.3 Fidelity is not relabelled accuracy

In every row of Table 2 fidelity and accuracy sit within about one point of each other, which might suggest fidelity adds nothing. With a 94.33%-accurate victim, any accurate model is necessarily a fairly faithful one, so the question worth asking is whether the substitute agrees with the victim more often than its accuracy alone would predict.

To test that we construct a null model. Suppose f_S and f_V err independently: f_S is correct with probability a , f_V with probability $v = 0.9433$, and each, when wrong, picks uniformly among the other nine classes. Expected agreement is then

$$F_{\text{indep}} = av + (1 - a)(1 - v) \cdot \frac{1}{9}. \quad (3)$$

Anything above this is evidence that the substitute has inherited the victim’s specific errors, which is what fidelity is meant to measure. Figure 5 plots the excess.

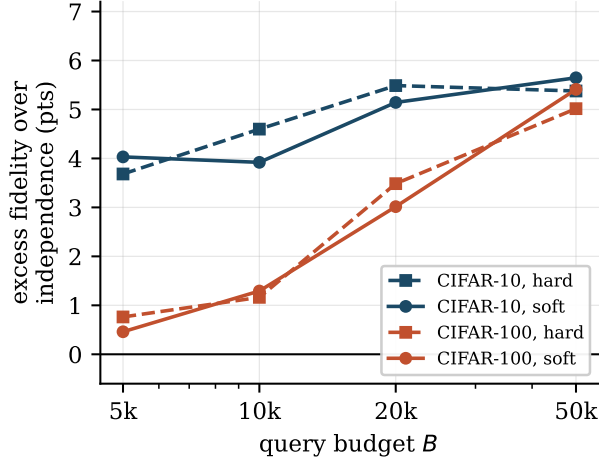


Figure 5: Measured fidelity minus the fidelity expected if the substitute’s errors were independent of the victim’s. The excess is positive everywhere and grows with budget, reaching +5.6 points for in-domain soft-label extraction at $B = 50,000$.

The excess is positive in all sixteen runs and rises with budget, from +0.5 points at $B = 5,000$ with CIFAR-100 queries to +5.6 points at $B = 50,000$ in-domain. At the largest budget an independent-errors model predicts 81.8% agreement; we observe 87.44%. Those extra 5.6 points are victim-specific idiosyncrasy that transferred through the API, and they are the part of the result that accuracy alone would have hidden.

This matters for the reconnaissance adversary of [3]. An extracted model is useful as a white-box stand-in for mounting transfer-based adversarial example attacks [4], and the rate at which those transfer depends on shared error structure rather than on the substitute’s standalone accuracy. Our measurements show that error structure is being copied, and that buying queries is the way to copy more of it: the excess grows with budget across every configuration.

At the two smallest CIFAR-100 budgets fidelity falls very slightly below accuracy, by 0.19 and 0.15 points. With a substitute this weak, occasional lucky agreement with ground truth on inputs the victim itself gets wrong is enough to invert the usual ordering. Both figures are well inside noise, but they mark the regime where the substitute has not yet begun tracking the victim specifically.

6 Defense Evaluation

6.1 What the server actually returns

Our design report nominated reducing output granularity as the main defense, with output perturbation listed as an alternative. All four policies are implemented in `_apply_defense` in `server.py` and selected at launch time through environment variables. They transform the response only; the underlying inference is identical in every case. Table 4 states precisely what each one emits, because the exact form of the returned vector turns out to explain the results.

Policy	Setting	Response
none	—	Clean label and full probability vector.
round	1 decimal	Label, and probabilities rounded to one decimal place. Not renormalised, so the vector need not sum to 1.
topk	$k = 1$	Label, and a vector that is zero everywhere except at the <code>argmax</code> , which keeps its true probability $p_{\max}$. Not renormalised, so the vector sums to $p_{\max}$.
label_only	—	Label only; the <code>probabilities</code> field is omitted.
noise	$\sigma = 0.1$	Clean label, and probabilities with $\mathcal{N}(0, 0.1^2)$ noise added per coordinate, clipped at zero and renormalised to sum to 1.

Table 4: What each output policy returns. All defense runs use the CIFAR-100 transfer pool. The attacker uses the soft loss under every policy except `label_only`, where no probabilities are available and the driver falls back to the hard loss.

6.2 Establishing a noise floor first

Before reading anything into the defense numbers we needed to know how large a difference has to be to mean anything. The undefended CIFAR-100 soft-label configuration was run twice, once in the main sweep and once as the `none` (`soft`) control in the defense sweep, which gives a direct estimate. Table 5 compares them.

Budget B	Run 1 (main sweep)	Run 2 (defense control)	Difference
5,000	20.32%	23.56%	3.24 pts
10,000	30.36%	30.82%	0.46 pts
20,000	48.18%	50.42%	2.24 pts
50,000	74.29%	74.99%	0.70 pts

Table 5: The same configuration, trained twice. The spread is our empirical noise floor: up to 3.24 points at small budgets, shrinking to 0.70 points at $B = 50,000$.

The two runs did not merely use similar transfer sets, they used identical ones. Both fix the pool ordering under seed 0, both query the same deterministic victim under the `none` policy, and the caching layer replays the same nested prefixes. The only difference between Run 1 and Run 2 is the unseeded randomness inside substitute training: weight initialisation, batch shuffling, dropout, and the random crops and flips applied to the queried images. The spread in Table 5 is therefore a clean measurement of substitute-training variance, with the query-side contribution removed by construction.

This sets the bar for every claim that follows. Any defense effect smaller than roughly 3 points at low budgets, or 1 point at 50,000 queries, is not distinguishable from re-running the same experiment. The estimate comes from one paired repetition rather than a proper variance over seeds, and a rigorous version of this study would run three to five seeds per cell.

6.3 Defense results

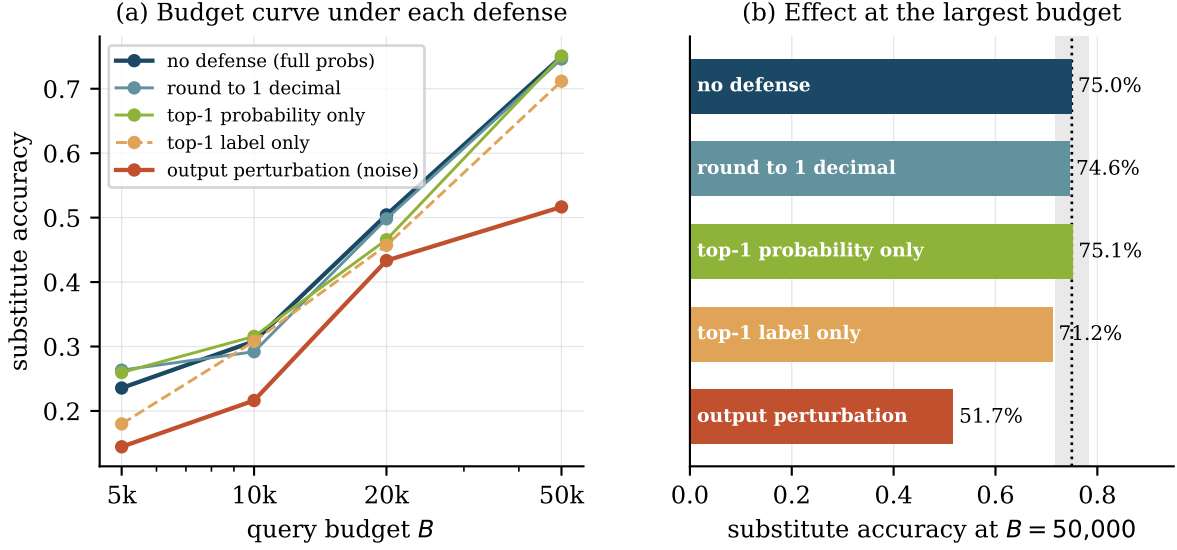


Figure 6: Defense evaluation, all with CIFAR-100 transfer queries. (a) Budget curves under each policy. (b) Substitute accuracy at $B = 50,000$; the dotted line is the undefended control and the grey band is the ± 3.24 -point noise floor from Table 5. Only output perturbation escapes the band.

Policy	Substitute accuracy at budget B			
	5,000	10,000	20,000	50,000
None (full probabilities)	23.56%	30.82%	50.42%	74.99%
Round to 1 decimal	26.33%	29.19%	49.82%	74.62%
Top-1 probability only	25.96%	31.57%	46.57%	75.08%
Top-1 label only	17.99%	30.80%	45.70%	71.17%
Output perturbation	14.44%	21.62%	43.34%	51.66%
<i>Change vs. no defense, percentage points (negative means the defense helped)</i>				
Round to 1 decimal	+2.77	-1.63	-0.60	-0.37
Top-1 probability only	+2.40	+0.75	-3.85	+0.09
Top-1 label only	-5.57	-0.02	-4.72	-3.82
Output perturbation	-9.12	-9.20	-7.08	-23.33

Table 6: Defense effectiveness. Differences below roughly 3 points at low budgets and 1 point at $B = 50,000$ are within run-to-run noise (Table 5).

Our main defense failed. Rounding probabilities to one decimal changed the substitute’s accuracy by -0.37 points at the largest budget. Returning only the top-1 probability changed it by $+0.09$ points, leaving the attacker marginally better off. Both effects sit far inside the noise floor, and at several budgets these policies appear to help the attacker, which is what measuring nothing looks like. Reducing the precision of the returned confidences, the countermeasure we designated as our main defense, is indistinguishable from doing nothing.

Label-only access is a speed bump. Withholding confidences entirely is the strongest form of granularity reduction and the only one of the three that registers: -3.82 points at $B = 50,000$, against a 0.70 -point noise floor at that budget. The effect is real but small. The attacker still reaches 71.17% accuracy, 75.4% of the victim’s capability, with nothing but integers coming back from the API. This is the warning Tramèr et al. [1] issued and the access restriction Jagielski et al. [3] describe, and our

numbers price it: roughly four accuracy points, paid for by degrading the API for every honest client that needs confidence values.

Output perturbation is the only policy that bites. Adding noise to the returned probabilities cut the substitute from 74.99% to 51.66% at 50,000 queries, a 23.33-point drop, and held it below 52% at every budget. Reading Figure 6(a) horizontally makes the cost concrete: the 51.66% the attacker reaches with 50,000 perturbed queries is what an undefended API would have surrendered in about 21,000 queries, so perturbation multiplies the attacker’s query cost by roughly 2.39 times. It is also the only curve whose gap to the control widens with budget rather than closing. Section 6.5 argues that this advantage does not survive a one-line change on the attacker’s side.

6.4 Why the granularity policies fail

The null results in Table 6 are not a coincidence of tuning. For two of the three policies, the soft-label loss the attacker runs reduces algebraically to the hard-label loss it would have run anyway.

Top-1 probability is confidence-weighted hard cross-entropy. Under `topk` with $k = 1$, the returned vector p is zero everywhere except at the victim’s `argmax` c , where $p_c = p_{\max}$. Substituting into the soft loss gives

$$\mathcal{L} = - \sum_j p_j \log \text{softmax}(z)_j = -p_{\max} \log \text{softmax}(z)_c = p_{\max} \cdot \text{CE}(z, c). \quad (4)$$

The attacker is running ordinary cross-entropy on the victim’s top-1 label, weighted per example by the victim’s confidence in it. The defense does not remove the attacker’s signal; it reweights it.

That reweighting is not neutral, and on out-of-distribution queries it favours the attacker. Examples where the victim is unsure receive a small weight and contribute little, which is a free sample-quality filter over exactly the noisy, ambiguous responses that CIFAR-100 probes elicit. The measurements agree: truncating to the top-1 probability beats withholding confidences altogether at all four budgets, by +7.97, +0.77, +0.87 and +3.91 points respectively. Returning the top-1 probability is therefore worse for the defender than returning no probability at all, because the confidence value doubles as a usable weight.

Rounding preserves what matters at this granularity. Rounding to one decimal maps any coordinate below 0.05 to zero and any coordinate above 0.95 to one. For the confident, near-one-hot outputs a victim produces on in-distribution images, the rounded vector is exactly one-hot and the soft loss again collapses to hard cross-entropy. For the diffuse outputs CIFAR-100 probes elicit, an 11-level quantisation at 0.1 spacing still preserves the ordering and rough shape of the top few coordinates, which is the part the substitute learns from. Since our defense sweep runs entirely on CIFAR-100, rounding lands in the second case and costs the attacker nothing measurable.

The common thread is that all three granularity policies remove information while leaving what remains correct. The attacker’s core signal is the location of the victim’s decision boundary, and `argmax` alone locates it. Our soft-versus-hard comparison in Table 3 already showed the probability tail is worth only a few accuracy points in this setting, so removing it cannot cost much.

6.5 The perturbation result does not survive an adaptive attacker

Output perturbation differs in kind from the granularity policies. It does not abbreviate the response, it corrupts it. To characterise how, we applied the exact transform from `_apply_defense` to synthetic victim outputs at a range of confidence levels, 200,000 draws each. Table 7 reports what it does.

Clean $p_{\max}$	Mass left on that class after	argmax of returned vector flipped
0.99	0.742	0.0%
0.95	0.723	0.0%
0.80	0.644	0.0%
0.50	0.443	0.6%
0.35	0.321	13.2%

Table 7: Effect of the $\sigma = 0.1$ perturbation on a 10-class probability vector, measured by applying the server’s own transform to synthetic inputs. It strips roughly a quarter of the mass off the predicted class and scatters it across the others, while rarely changing which class is largest unless the victim was already uncertain.

The mechanism is mass redistribution rather than label corruption. A confident response keeps its `argmax` but loses about a quarter of its probability mass to random coordinates, which destroys precisely the similarity structure the soft loss reads. On diffuse out-of-distribution responses the perturbation also begins flipping the `argmax`, which is why the effect is large in our CIFAR-100 sweep.

There is a flaw in our implementation, and it is decisive. `_apply_defense` perturbs the probability vector but returns the *clean* `argmax` in the `labels` field, by design, so that honest clients reading only the label are unaffected. Our attacker logs both fields and then, because it is configured with the soft loss, trains on the corrupted vector and ignores the uncorrupted label sitting beside it. An attacker who instead ran the hard loss under this policy would be training on labels identical to the undefended ones, and would land at approximately the `label_only` result, 71.17%, rather than 51.66%.

The 23-point drop we measured is therefore the cost of our attacker’s fixed configuration, not a property of the defense. Against an adversary that selects the stronger of the two fields it receives, our perturbation policy is worth about as much as label-only access, which is to say about four points. The concept from Orekondy et al. [7] is not what fails here; our implementation is. A correct one must ensure the returned label is the `argmax` of the returned vector, so that the perturbation cannot be bypassed by reading a different field. That change would also move the honest-utility cost from confidence consumers onto label consumers, which is the trade-off a real operator has to price, and which our current implementation avoids only by leaving the attack intact.

Taken together, none of the four policies we implemented stops an adaptive attacker. The best any of them achieves against one is the roughly four-point reduction from withholding confidences entirely.

7 Comparison with Our Design-Report Predictions

We stated four expectations on 11 September. Table 8 audits them.

We predicted	We measured
f_S approaches f_V ’s accuracy using far fewer queries than the victim’s training set size.	Partly. f_S reaches 91.8% of f_V ’s accuracy, but only at $B = 50,000$, which equals the victim’s training set size exactly. At genuinely small budgets the gap is wide.
Agreement rises with budget and plateaus, showing high fidelity.	Rises, but no plateau. Fidelity climbs monotonically to 87.44% and had not flattened at our largest budget. The plateau predicted analytically by [3] lies beyond our sweep.
The attack succeeds without the victim’s architecture or training data.	Confirmed. 74.29% accuracy from CIFAR-100 queries alone, with no ground-truth labels and a substitute from a different architecture family.
f_S within a few percentage points of f_V at a modest budget using OOD data.	Not met. The gap at $B = 50,000$ with OOD data is 20.0 points.

Table 8: Design-report expectations against measured outcomes.

The fourth prediction is the one we got wrong, and the reason is mostly architectural. We wrote the prediction with Knockoff Nets [2] in mind, but that work reports its strongest out-of-distribution results with knockoff networks initialised from ImageNet-pretrained weights and transfer pools drawn from ImageNet. Our substitute is a 2.36M-parameter four-layer CNN trained from random initialisation on at most 50,000 noisy targets, against an 11.17M-parameter ResNet-18 victim trained on 50,000 clean ones. It has to learn a representation and a decision boundary simultaneously, from a weaker function class, and with no ground truth.

This reframes the 86.62% in-domain ceiling. That number should be compared against what `SmallCNN` can achieve at all, not against the victim. We did not run that control, and it is the first thing we would add: train `SmallCNN` on the CIFAR-10 training split with ground-truth labels under identical hyperparameters, and report extraction accuracy as a fraction of *that* number rather than of the victim’s. Our expectation is that the ceiling is largely architectural and that the extraction ratio against a matched-capacity reference is considerably higher than 0.918, but we cannot claim it from the data we have. The second experiment would swap the substitute to `resnet18`, which `models.py` already supports, to separate capacity limits from query limits directly.

Our defense prediction was also wrong, and more interestingly so. We wrote that reducing output granularity “removes the soft-label signal and measurably increases the query budget the attacker needs.” Section 6 shows it does not, and the algebra explains why it could not. The defense we listed third among the alternatives appeared to be the only effective one, until Section 6.5 showed that its apparent strength came from a leak in our own server.

8 Limitations

- **Single seed per configuration.** Every cell in Tables 2 and 6 is one training run. Our noise floor comes from a single paired repetition (Table 5), not a variance over seeds. Differences of a few points should be read as suggestive only.
- **Budget ceiling.** No curve had plateaued at 50,000 queries, which is also the full size of each transfer pool, so every number here is a lower bound on the attack’s strength and we cannot probe further without a larger pool.
- **Substitute capacity is a confound.** `SmallCNN` is 4.7 times smaller than the victim and we never measured its ceiling on clean data, so we cannot separate the query limit from the architecture limit. Section 7 describes the control that would.
- **Training compute is not held constant per example.** The substitute trains for 40 epochs at every budget, so a 5,000-query run sees 200,000 sample presentations while a 50,000-query run sees two million. A real attacker would train longer on small transfer sets, so our low-budget points probably understate what the attack achieves there.
- **One victim.** A single architecture at a single accuracy level. Extraction difficulty depends on victim capacity [3], and we sampled one point.
- **Low-resolution task.** 32×32 ten-class classification is the easy end of the problem, though [3] reports extraction succeeding against production-scale systems.
- **Defenses tested against one fixed attacker.** Our attacker’s loss is chosen per policy by the driver and never adapts. Section 6.5 shows how much this matters for the perturbation result. An attacker that also averaged repeated queries could cancel the zero-mean noise directly, which our nested caching prevented us from testing.
- **Nested, non-independent budgets.** Budgets are prefixes of one shuffled ordering, so points on a curve share queries and their errors are correlated. Under the `noise` policy the cached perturbation draw is reused across budgets as well.
- **Random sampling only.** We did not implement the adaptive, reinforcement-learning query policy of Knockoff Nets [2], which selects informative images rather than sampling uniformly and would shift every curve leftward.

- **Honest-client utility unmeasured.** We report what each policy costs the attacker and never what it costs legitimate users. Under our perturbation implementation that cost falls entirely on clients that read confidence values, since the label is returned unperturbed; a corrected implementation would move part of it onto label consumers.

9 Conclusion

We built a working model-extraction attack and measured what it costs. Against a 94.33%-accurate CIFAR-10 ResNet-18 exposed through a prediction API, 50,000 queries recover a four-layer CNN at 86.62% accuracy and 87.44% agreement in-domain, and 74.29% accuracy using query images drawn from an entirely different label space, without the victim’s weights, architecture, training data, or a single ground-truth label. The substitute inherits victim-specific errors well beyond what its accuracy alone would explain, so fidelity and not just accuracy is being stolen. How much of the remaining gap is a query limit rather than a limit of the small substitute architecture is a question our experiments cannot answer, and we have described the control that would settle it.

The defense findings are the part we did not expect. Reducing output granularity, our designated main defense and the intuitive first move for any API operator, was indistinguishable from no defense. Rounding cost the attacker 0.37 points. Truncating to the top-1 probability cost nothing, and in fact left the attacker ahead of the label-only baseline at all four budgets, because under the soft loss that truncated vector is exactly a confidence-weighted hard label. Even withholding confidences entirely, the maximal version of the strategy, cost 3.82 points while the attacker still took 75% of the victim’s capability.

Output perturbation appeared to be the exception, with a 23-point drop and a 2.39-fold increase in query cost. Reading our own server code removed that conclusion. The policy returns a clean top-1 label alongside the corrupted probability vector, and our attacker trained on the corrupted field while the uncorrupted one sat in the same response. An attacker that read the other field would have landed near the label-only result. We report the 23-point figure because it is what we measured, and we retract the interpretation, because a defense that is bypassed by reading a different key in the same JSON object is not a defense.

What survives is a narrower and more useful claim than the one we set out to make. Defenses that reduce what a prediction API says do not work against an adversary that only needs to know where the decision boundary lies, and a defense that corrupts what the API says only works if every field it returns is corrupted consistently.

References

- [1] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, “Stealing Machine Learning Models via Prediction APIs,” in *Proc. 25th USENIX Security Symposium*, Austin, TX, USA, Aug. 2016, pp. 601–618.
- [2] T. Orekondy, B. Schiele, and M. Fritz, “Knockoff Nets: Stealing Functionality of Black-Box Models,” in *Proc. IEEE/CVF CVPR*, Long Beach, CA, USA, Jun. 2019, pp. 4954–4963.
- [3] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, and N. Papernot, “High Accuracy and High Fidelity Extraction of Neural Networks,” in *Proc. 29th USENIX Security Symposium*, Aug. 2020, pp. 1345–1362.
- [4] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, “Practical Black-Box Attacks against Machine Learning,” in *Proc. ACM ASIA CCS ’17*, Abu Dhabi, UAE, Apr. 2017, pp. 506–519.
- [5] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NIPS 2014 Deep Learning Workshop*, 2015, arXiv:1503.02531.
- [6] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, “PRADA: Protecting Against DNN Model Stealing Attacks,” in *Proc. IEEE EuroS&P*, Stockholm, Sweden, Jun. 2019, pp. 512–527.
- [7] T. Orekondy, B. Schiele, and M. Fritz, “Prediction Poisoning: Towards Defenses Against DNN Model Stealing Attacks,” in *Proc. ICLR*, Apr. 2020.

- [8] J. R. Correia-Silva, R. F. Berriel, C. Badue, A. F. De Souza, and T. Oliveira-Santos, “Copycat CNN: Stealing Knowledge by Persuading Confession with Random Non-Labeled Data,” in *Proc. IJCNN*, Rio de Janeiro, Brazil, Jul. 2018, pp. 1–8.
- [9] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Technical Report, Dept. of Computer Science, University of Toronto, 2009.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE CVPR*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.

A Query Cost at Fixed Accuracy Targets

Table 9 inverts the budget curves. Instead of asking what accuracy a budget buys, it asks what budget an accuracy target costs. Values are log-linear interpolations between measured points. This is the natural way to express a defense’s strength, since a defender cares about raising the attacker’s bill rather than about accuracy points as such.

Configuration	30% acc.	50% acc.	70% acc.
CIFAR-10 pool, hard labels	< 5k	< 5k	≈ 11,900
CIFAR-10 pool, soft labels	< 5k	< 5k	≈ 14,300
CIFAR-100 pool, hard labels	≈ 10,500	≈ 19,200	> 50k
CIFAR-100 pool, soft labels	≈ 9,800	≈ 21,300	≈ 43,000
<i>Under each output policy (CIFAR-100 pool)</i>			
None	≈ 9,200	≈ 19,700	≈ 41,500
Round to 1 decimal	≈ 10,300	≈ 20,100	≈ 42,200
Top-1 probability only	≈ 8,200	≈ 22,300	≈ 42,500
Top-1 label only	≈ 9,600	≈ 23,300	≈ 47,900
Output perturbation	≈ 13,100	≈ 41,600	> 50k

Table 9: Queries required to reach each accuracy target. The perturbation row applies only to our non-adaptive attacker; see Section 6.5.

B Reproducing the Analysis

All tables and figures in this report derive from two files: `main_results.csv` (columns `transfer`, `queries`, `loss`, `substitute_accuracy`, `victim_accuracy`, `fidelity`; 16 rows) and `defense_results.csv` (the same columns plus `defense`; 20 rows). The derived quantities are:

- $\text{extraction ratio} = \text{substitute_accuracy} / 0.9433$;
- $\text{excess fidelity} = \text{fidelity} - [av + (1 - a)(1 - v)/9]$, with $a = \text{substitute_accuracy}$ and $v = 0.9433$;
- query cost by log-linear interpolation of accuracy against $\log_{10} B$.

Table 7 is the one figure not derived from the results files. It was produced by applying the `noise` branch of `_apply_defense` verbatim to 200,000 synthetic 10-class probability vectors at each listed confidence level, with the remaining mass spread evenly over the other nine classes. It characterises the transform, not the victim.